

Speaking Feature - GitHub Issue Draft

File này chứa nội dung để copy-paste vào GitHub Issues. Mỗi section tương ứng với 1 issue trên GitHub. Xem hướng dẫn sử dụng GitHub Issues ở cuối file.

PARENT ISSUE

Copy phần **Title** và **Body** bên dưới vào issue chính trên GitHub.

Title

Feature: Tính năng Speaking - AI IELTS Examiner

Labels

speaking, feature, priority:high

Body

---BEGIN COPY---

Tổng quan

Xây dựng tính năng **IELTS Speaking Practice** với AI đóng vai examiner, sử dụng Gemini Live API cho real-time audio hai chiều.

Kiến trúc 3 tầng:

- **Frontend**: Giao diện thi, thu âm, hiển thị transcript real-time
- **Backend REST**: Quản lý session, lưu trữ, trigger chấm điểm
- **Backend WebSocket**: Luồng audio real-time hai chiều qua Gemini Live API

Tài liệu tham khảo

- Design doc gốc: [docs/short_term_plans/speaking_issue.md](#)
- Foundation spec: [docs/ielts_specific/speaking_session_foundations_vi.md](#)
- [Gemini Live API docs](#)

Lưu ý quan trọng

Đây là **viết lại hoàn toàn** (rewrite). Code Speaking cũ trên **main** (từ branch **feature/speaking-gemini-live**) sẽ được xóa. DB tables cũ sẽ được drop và tạo lại với schema đã chỉnh sửa.

Các chỉnh sửa so với design doc gốc (đã validate với codebase):

- 1. `is_finalized` nằm trên **session**, KHÔNG phải trên transcript
- 2. `CreateSpeakingSessionDTO` cần bao gồm `accent` và `speed` (đọc đúng, code cũ thiếu)
- 3. `GET /sessions/{id}/grading-status` là endpoint mới, code cũ không có
- 4. `SpeakingAudioCleanupJob` chưa từng tồn tại, cần implement mới
- 5. Lúa được **kiểm tra** khi tạo session, nhưng **trừ** khi hoàn thành (abandon = không mất lúa)

Luồng chính (User Flow)

- 1. **Chọn bài thi:** User chọn "Test Speaking" -> Modal chọn Part (1/2/3) hoặc Full Test
- 2. **Chuẩn bị:** Cấp quyền mic, chọn accent & speed, xem thông tin buổi thi
- 3. **Thi:** Câu hỏi text + AI đọc audio -> User trả lời -> Transcript real-time
- 4. **Chờ chấm:** Loading screen trong khi AI grading (async)
- 5. **Kết quả:** Hiển thị điểm chi tiết theo 4 tiêu chí IELTS

Cơ chế tín dụng (Lúa)

Thời điểm	Hành động
Tạo session (<code>POST /api/speaking/sessions</code>)	Kiểm tra số dư, chưa trừ
Hoàn thành (<code>POST /sessions/{id}/complete</code>)	Trừ lúa nếu <code>lua_deducted = false</code>
Bỏ dở (<code>POST /sessions/{id}/abandon</code>)	KHÔNG trừ lúa

Sub-issues

- ☐ #__ -- [DB] Thiết kế schema & migration cho Speaking
- ☐ #__ -- [BE] REST API - Quản lý session Speaking
- ☐ #__ -- [BE] WebSocket + Gemini Live API integration
- ☐ #__ -- [BE] Hệ thống chấm điểm (Grading & Evaluation)
- ☐ #__ -- [FE] Core flow - Pages, routing, state management
- ☐ #__ -- [FE] Real-time session - Audio & WebSocket
- ☐ #__ -- [Infra] Cleanup job, fallback & kiểm thử

Thay #__ bằng số issue thật sau khi tạo từng sub-issue trên GitHub.

Sequence Diagram

```
sequenceDiagram
    autonumber
    participant U as User
    participant FE as Frontend
    participant API as Backend REST
    participant WS as Backend WS
    participant GL as Gemini Live API
    participant SB as Supabase Storage
    participant EV as EvalService
    participant OR as OpenRouter

    U->>FE: Chọn test/part và Start
```

```
FE->>API: POST /api/speaking/sessions
Note over API: Kiểm tra lúa (chưa trừ)
API-->>FE: sessionId + question set

FE->>WS: Connect /ws/speaking/{sessionId}
WS->>WS: Check speaking.gemini.live.enabled

alt Gemini Live configured
  WS->>GL: Open WebSocket (?key=GEMINI_API_KEY)
  GL-->>WS: setupComplete
  WS-->>FE: status=ready

  loop Mỗi câu hỏi
    FE->>WS: type=start_question
    WS->>GL: sendTextMessage(question prompt)
    GL-->>WS: examiner audio + transcript
    WS-->>FE: examiner_audio + transcript

    FE->>WS: binary audio chunks (user mic)
    WS->>GL: realtime_input.media_chunks
    GL-->>WS: inputTranscript + turnComplete
    WS-->>FE: user transcript + speaking state

    FE->>SB: Upload answer audio
    SB-->>FE: public audio URL
    FE->>API: POST /sessions/{id}/transcripts
    API-->>FE: saved
  end
else Gemini Live not configured
  WS-->>FE: status=fallback_text_mode
end

FE->>API: POST /sessions/{id}/complete
Note over API: Trừ lúa tại đây
API->>EV: Trigger async grading

EV->>SB: Download audio từ URLs
EV->>OR: callWithAudio (Gemini model)
OR-->>EV: JSON grading result
EV->>API: Save scores + status=completed

FE->>API: GET /sessions/{id}/grading-status (polling)
API-->>FE: grading/completed
FE->>API: GET /sessions/{id}/results
API-->>FE: Kết quả chi tiết
```

Cơ chế Fallback

Tình huống	Fallback
Gemini Live không kết nối được	<code>status=fallback_text_mode</code> -- FE hiển thị câu hỏi text, user trả lời bình thường

Tình huống	Fallback
Gemini phản hồi chậm (>3s)	FE tự động mở mic cho user trả lời dựa trên text
Upload audio / lưu transcript lỗi	Try-catch per question, session tiếp tục
ASR lỗi (transcript rỗng liên tục)	TODO - cần implement

---END COPY---

SUB-ISSUE 1: [DB] Thiết kế schema & migration cho Speaking

Title

[DB] Thiết kế schema & migration cho Speaking

Labels

speaking, database, priority:high

Body

---BEGIN COPY---

Mô tả

Drop toàn bộ speaking tables cũ và tạo lại schema mới với các chỉnh sửa đã xác nhận.

Parent issue: #__

Bước đầu tiên: Xóa tables cũ

```
DROP TABLE IF EXISTS speaking_transcripts CASCADE;
DROP TABLE IF EXISTS speaking_sessions CASCADE;
DROP TABLE IF EXISTS speaking_fixed_questions CASCADE;
DROP TABLE IF EXISTS speaking_questions CASCADE;
DROP TABLE IF EXISTS speaking_tests CASCADE;
DROP TABLE IF EXISTS speaking_topics CASCADE;
```

Các bảng cần tạo

1. speaking_topics

Column	Type	Nullable	Default	Mô tả
id	BIGSERIAL	NO	auto	PK
code	VARCHAR	NO		Mã topic (vd: music , travel)
name_vi	VARCHAR	NO		Tên tiếng Việt
name_en	VARCHAR	NO		Tên tiếng Anh
icon	VARCHAR	YES		Emoji/icon
color	VARCHAR	YES		Màu hiển thị
is_active	BOOLEAN	YES	true	Còn hoạt động
created_at	TIMESTAMPTZ	YES	now()	
updated_at	TIMESTAMPTZ	YES	now()	

2. **speaking_tests**

Column	Type	Nullable	Default	Mô tả
id	BIGINT	NO		PK
topic_id	BIGINT	NO		FK -> speaking_topics
test_number	INTEGER	NO		Số thứ tự test
name	VARCHAR	YES		Tên test
description	TEXT	YES		Mô tả
max_questions	INTEGER	NO	30	Số câu tối đa
difficulty	VARCHAR	YES	'INTERMEDIATE'	Độ khó
is_active	BOOLEAN	YES	true	
created_at	TIMESTAMPTZ	YES	now()	
updated_at	TIMESTAMPTZ	YES	now()	

3. **speaking_questions**

Column	Type	Nullable	Default	Mô tả
id	BIGSERIAL	NO	auto	PK
topic_id	BIGINT	YES		FK -> speaking_topics
test_id	BIGINT	YES		FK -> speaking_tests
part	INTEGER	NO		Part 1/2/3
question_text	TEXT	NO		Nội dung câu hỏi

Column	Type	Nullable	Default	Mô tả
cue_card_bullets	JSONB	YES		Bullet points cho Part 2
difficulty	VARCHAR	YES		Độ khó
register	VARCHAR	YES		Formal/informal
expected_length_seconds	INTEGER	YES		Thời gian trả lời dự kiến
follow_up_allowed	BOOLEAN	YES	false	Cho phép follow-up
follow_up_question_ids	BIGINT[]	YES		IDs câu follow-up
examiner_audio_url	VARCHAR	YES		URL audio TTS sẵn
examiner_audio_duration_ms	INTEGER	YES		Độ dài audio TTS
tts_voice_id	VARCHAR	YES	'Rachel'	Voice TTS
tts_generated_at	TIMESTAMPTZ	YES		Thời điểm gen TTS
version	INTEGER	YES	1	
is_active	BOOLEAN	YES	true	
created_at	TIMESTAMPTZ	YES	now()	
updated_at	TIMESTAMPTZ	YES	now()	

4. speaking_fixed_questions

Câu hỏi cố định (warm-up Part 1, etc.)

Column	Type	Nullable	Default	Mô tả
id	BIGINT	NO		PK
question_order	INTEGER	NO		Thứ tự câu hỏi
question_text	TEXT	NO		Nội dung
question_text_vi	TEXT	YES		Bản dịch
question_purpose	VARCHAR	NO		Mục đích (warm-up, main, etc.)
is_active	BOOLEAN	YES	true	
created_at	TIMESTAMPTZ	YES	now()	

5. speaking_sessions

Column	Type	Nullable	Default	Mô tả
id	BIGSERIAL	NO	auto	PK
user_id	UUID	NO		FK -> auth.users

Column	Type	Nullable	Default	Mô tả
session_mode	VARCHAR(20)	NO		full, part1, part2, part3
topic_id	BIGINT	YES		FK -> speaking_topics
test_id	BIGINT	YES		FK -> speaking_tests
status	VARCHAR(20)	NO	'in_progress'	Xem bảng status bên dưới
accent	VARCHAR	YES	'neutral'	Giọng AI examiner
speed	VARCHAR	YES	'normal'	Tốc độ đọc
session_blueprint	JSONB	YES		Blueprint câu hỏi cho session
is_finalized	BOOLEAN	YES	false	Session đã submit chính thức chưa
started_at	TIMESTAMPTZ	YES	now()	
completed_at	TIMESTAMPTZ	YES		
total_duration_seconds	INTEGER	YES		
overall_band	NUMERIC(2,1)	YES		Điểm tổng
fluency_band	NUMERIC(2,1)	YES		Fluency & Coherence
lexical_band	NUMERIC(2,1)	YES		Lexical Resource
grammar_band	NUMERIC(2,1)	YES		Grammatical Range
pronunciation_band	NUMERIC(2,1)	YES		Pronunciation
grading_result	JSONB	YES		Chi tiết chấm điểm
graded_at	TIMESTAMPTZ	YES		Thời điểm chấm xong
lua_cost	INTEGER	NO	0	Chi phí lua
lua_deducted	BOOLEAN	YES	false	Đã trừ lua chưa
created_at	TIMESTAMPTZ	YES	now()	
updated_at	TIMESTAMPTZ	YES	now()	

Session status flow:

```
in_progress -> completed -> grading -> graded
                                     \-> grading_failed
in_progress -> abandoned
in_progress -> expired (bị cleanup job dọn)
```

LƯU Ý: is_finalized nằm trên **session**, KHÔNG phải trên transcript.

6. speaking_transcripts

Column	Type	Nullable	Default	Mô tả
id	BIGSERIAL	NO	auto	PK
session_id	BIGINT	NO		FK -> speaking_sessions
question_id	BIGINT	YES		FK -> speaking_questions
part	INTEGER	NO		Part 1/2/3
audio_url	TEXT	YES		URL audio trả lời
audio_duration_seconds	INTEGER	YES		Độ dài audio
transcript_text	TEXT	YES		Nội dung transcript
transcript_confidence	NUMERIC(3,2)	YES		Độ tin cậy ASR
recorded_at	TIMESTAMPTZ	YES	now()	
question_evaluation	JSONB	YES		Đánh giá per-question
created_at	TIMESTAMPTZ	YES	now()	

Checklist

- ☐ Viết migration SQL: DROP existing 6 tables (CASCADE)
- ☐ Viết migration SQL: CREATE 6 tables mới
- ☐ Thêm foreign key constraints và indexes
- ☐ Thiết lập RLS policies cho `speaking_sessions` và `speaking_transcripts` (user chỉ xem được của mình)
- ☐ Seed data: thêm ít nhất 3 topics với 5-10 câu hỏi mỗi part
- ☐ Tạo Supabase Storage bucket `speaking-audio` (nếu chưa có)
- ☐ Viết migration file vào `docs/backend/migrations/`

Acceptance Criteria

- ☐ Tất cả 6 bảng tồn tại trong Supabase với schema đúng
- ☐ RLS policies hoạt động đúng (user A không xem được session user B)
- ☐ Có seed data để test
- ☐ Storage bucket `speaking-audio` tồn tại

---END COPY---

SUB-ISSUE 2: [BE] REST API - Quản lý session Speaking

Title

[BE] REST API - Quản lý session Speaking

Labels


```
speaking, backend, priority:high
```

Body

---BEGIN COPY---

Mô tả

Xây dựng REST API layer cho Speaking feature: controller, services, DTOs, repositories.

Parent issue: #__ **Depends on:** [DB] Schema & migration (#__)

Endpoints

Session Management

Method	Path	Mô tả
POST	/api/speaking/sessions	Tạo session mới
GET	/api/speaking/sessions/{id}	Lấy thông tin session
POST	/api/speaking/sessions/{id}/transcripts	Lưu transcript cho 1 câu
POST	/api/speaking/sessions/{id}/complete	Submit bài, trigger grading
POST	/api/speaking/sessions/{id}/abandon	Hủy session (không trừ lúa)
GET	/api/speaking/sessions/{id}/grading-status	Poll trạng thái chấm điểm
GET	/api/speaking/sessions/{id}/results	Lấy kết quả chi tiết
GET	/api/speaking/history	Lịch sử thi của user

Content

Method	Path	Mô tả
GET	/api/speaking/topics	Danh sách topics
GET	/api/speaking/questions	Câu hỏi theo topic & mode

Chi tiết API

POST /api/speaking/sessions

Request body (CreateSpeakingSessionDTO):

```
{
  "mode": "full | part1 | part2 | part3",
  "topicId": 1,
```

```

    "accent": "british | american | australian | neutral",
    "speed": "slow | normal | fast"
  }

```

accent và **speed** **phải có** trong DTO. Design doc gốc đúng, code cũ thiếu 2 field này.

Logic tuần tự:

1. Validate **mode** (giá trị hợp lệ: **full**, **part1**, **part2**, **part3**)
2. Kiểm tra lúa đủ không (**creditService.hasEnoughCredits**)
3. Tạo session trong DB (**status** = **"in_progress"**, **lua_deducted** = **false**)
4. Random câu hỏi phù hợp topic/mode
5. Trả về **sessionId** + question set

Response: **SpeakingSessionDTO**

```

{
  "sessionId": 42,
  "sessionMode": "full",
  "topicId": 1,
  "status": "in_progress",
  "luaCost": 15,
  "accent": "british",
  "speed": "normal",
  "questions": [ ... ]
}

```

POST /api/speaking/sessions/{id}/transcripts

Request body:

```

{
  "questionId": 5,
  "part": 1,
  "audioUrl": "https://...supabase.co/storage/v1/object/...",
  "transcriptText": "I think music is very important...",
  "audioDurationSeconds": 45
}

```

POST /api/speaking/sessions/{id}/complete

Logic tuần tự:

1. Set **is_finalized** = **true**
2. Set **status** = **"grading"**
3. Tính **total_duration_seconds** (từ **started_at** đến now)
4. **Trừ lúa** (**creditService.spendCredits**) -- chỉ nếu **lua_deducted** = **false**

5. Set `lua_deducted = true`
6. Trigger async evaluation (xem sub-issue Grading)

Response: { "message": "Session completed. Evaluation in progress." }

POST `/api/speaking/sessions/{id}/abandon`

Logic:

1. Set `status = "abandoned"`
2. **KHÔNG** trừ lúa

GET `/api/speaking/sessions/{id}/grading-status` (MỚI)

Endpoint này **không tồn tại** trong code cũ. Cần implement mới.

Response:

```
{
  "status": "grading | graded | grading_failed",
  "progress": "Đang phân tích audio...",
  "estimatedSeconds": 30
}
```

GET `/api/speaking/sessions/{id}/results`

Response: `SpeakingResultDTO` (xem sub-issue Grading cho chi tiết)

Entities cần tạo

Entity	Table	Package
SpeakingSession	speaking_sessions	com.cramer.entity
SpeakingTranscript	speaking_transcripts	com.cramer.entity
SpeakingQuestion	speaking_questions	com.cramer.entity
SpeakingTopic	speaking_topics	com.cramer.entity

DTOs cần tạo

DTO	Mục đích
CreateSpeakingSessionDTO	Request tạo session (mode, topicId, accent, speed)
SpeakingSessionDTO	Response session info
SpeakingTranscriptDTO	Response transcript
SpeakingQuestionDTO	Response question

DTO	Mục đích
SpeakingTopicDTO	Response topic
SpeakingResultDTO	Response kết quả chấm điểm
SpeakingGradingResultDTO	Kết quả AI grading
SpeakingGradingStatusDTO	Trạng thái chấm điểm (MỚI)

Services cần tạo

Service	Trách nhiệm
SpeakingSessionService	Session lifecycle (create, complete, abandon)
SpeakingQuestionService	Lấy topics, questions, random selection

Repositories cần tạo

Repository	Entity
SpeakingSessionRepository	SpeakingSession
SpeakingTranscriptRepository	SpeakingTranscript
SpeakingQuestionRepository	SpeakingQuestion
SpeakingTopicRepository	SpeakingTopic

Configuration

```
speaking.session.lua-cost=${SPEAKING_LUA_COST:15}
```

Acceptance Criteria

- ☐ Tất cả endpoints hoạt động qua Swagger UI
- ☐ Credit check chặn session khi hết lúa (trả 400/403)
- ☐ Credit deduction chỉ xảy ra khi complete (không khi abandon)
- ☐ Grading-status endpoint trả đúng trạng thái real-time
- ☐ History endpoint phân trang đúng
- ☐ Auth: tất cả endpoints yêu cầu JWT token

---END COPY---

SUB-ISSUE 3: [BE] WebSocket + Gemini Live API Integration

Title

```
[BE] WebSocket + Gemini Live API integration
```

Labels

```
speaking, backend, priority:high
```

Body

---BEGIN COPY---

Mô tả

Xây dựng WebSocket layer kết nối Frontend <-> Backend <-> Gemini Live API cho real-time speaking session.

Parent issue: #__ **Depends on:** [BE] REST API (#__)

Kiến trúc

```
Browser <--WebSocket--> Spring Backend <--WebSocket--> Gemini Live API
                        /ws/speaking/{sessionId}
wss://generativelanguage.googleapis.com/ws/...
```

Lý do dùng WebSocket thay vì REST: REST request-response không phù hợp cho stream audio liên tục với độ trễ thấp. WebSocket duy trì kết nối hai chiều, cho phép gửi/nhận audio gần như đồng thời trong suốt buổi thi.

Components cần tạo

1. `WebSocketConfig` (`com.cramer.config`)

- Đăng ký `SpeakingWebSocketHandler` tại `/ws/speaking/{sessionId}`
- Cho phép CORS từ frontend origins
- Implement `WebSocketConfigurer`

```
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {
    @Override
    public void registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(speakingWebSocketHandler,
            "/ws/speaking/{sessionId}")
            .setAllowedOrigins("*");
    }
}
```

2. SpeakingWebSocketHandler (com.cramer.websocket)

Extends `AbstractWebSocketHandler`. Quản lý vòng đời WebSocket session.

Kết nối (afterConnectionEstablished):

- 1. Extract `sessionId` từ URI path
- 2. Kiểm tra `speaking.gemini.live.enabled` và `gemini.api.key`
- 3. Nếu OK -> tạo `GeminiLiveWebSocketClient`, kết nối tới Gemini
- 4. Nhận `setupComplete` từ Gemini -> gửi `status=ready` về Frontend
- 5. Nếu KHÔNG OK -> gửi `status=fallback_text_mode` về Frontend

Message protocol (Frontend -> Backend):

Type	Format	Mô tả
<code>start_question</code>	JSON <code>{"type": "start_question"}</code>	User sẵn sàng nhận câu tiếp
<code>end_turn</code>	JSON <code>{"type": "end_turn"}</code>	User kết thúc trả lời
binary frames	Binary	Audio chunks từ microphone

Message protocol (Backend -> Frontend):

Type	Format	Mô tả
<code>status</code>	<code>{"type": "status", "status": "ready"}</code>	Trạng thái kết nối
<code>examiner_audio</code>	Binary frames	Audio AI đọc câu hỏi
<code>transcript</code>	<code>{"type": "transcript", "text": "...", "source": "examiner\\ user"}</code>	Transcript real-time
<code>examiner_speaking</code>	<code>{"type": "examiner_speaking", "speaking": true\\ false}</code>	AI đang nói hay đã dừng
<code>turn_complete</code>	<code>{"type": "turn_complete"}</code>	Kết thúc lượt
<code>error</code>	<code>{"type": "error", "message": "..."} </code>	Lỗi

Disconnect (afterConnectionClosed):

- 1. Đóng Gemini WebSocket client
- 2. Dọn state nội bộ

3. GeminiLiveWebSocketClient (com.cramer.websocket)

Extends `org.java_websocket.client.WebSocketClient`. Kết nối tới Gemini Live API.

URI:

```
wss://generativelanguage.googleapis.com/ws/google.ai.generativelanguage.v1alpha
.GenerativeService.BidiGenerateContent?key=<API_KEY>
```

Lifecycle:

1. **onOpen**: Gửi setup message (model config, voice, system instruction)
2. **onMessage**: Parse Gemini responses (**modelTurn**, **inputTranscript**, **turnComplete**, **setupComplete**)
3. Forward audio/transcript về **SpeakingWebSocketHandler**
4. **onClose/onError**: Log + thông báo Frontend

Key methods:

- **sendSetupMessage()** -- cấu hình model, voice, system prompt
- **sendQuestionPrompt(String questionText)** -- gửi câu hỏi cho examiner đọc
- **sendAudioChunk(byte[] audioData)** -- forward audio user
- **sendEndOfTurn()** -- báo user nói xong
- **createGeminiUri(String apiKey)** -- static, build URI

4. **GeminiLiveService (com.cramer.service)** -- Optional helper

- Build prompt text với role examiner instructions
- Inject accent/speed settings vào prompt
- Quản lý connection pool nếu cần

Vòng lặp câu hỏi (per-question cycle)

1. FE gửi {type: "start_question"}
2. Backend gửi prompt tới Gemini (question text + examiner role)
3. Gemini phản hồi:
 - a. examiner_audio (binary) -> forward tới FE
 - b. transcript (JSON) -> forward tới FE
 - c. Khi đọc xong: examiner_speaking=false -> FE mở mic
4. User nói -> FE gửi binary audio chunks
5. Backend forward audio tới Gemini via `realtime_input.media_chunks`
6. Gemini trả về `inputTranscript` + `turnComplete`
7. Backend forward về FE
8. Lặp lại từ bước 1 cho câu tiếp theo

Configuration

```
speaking.gemini.live.enabled=${SPEAKING_GEMINI_LIVE_ENABLED:true}
gemini.api.key=${GEMINI_API_KEY:}
```

Dependencies (Maven)

```
<!-- Java-WebSocket client library -->
<dependency>
  <groupId>org.java-websocket</groupId>
  <artifactId>Java-WebSocket</artifactId>
  <version>1.5.6</version>
</dependency>
```

Acceptance Criteria

- ☐ WebSocket kết nối thành công FE <-> BE <-> Gemini
- ☐ Audio hai chiều hoạt động (examiner đọc câu hỏi, user trả lời)
- ☐ Transcript real-time hiển thị đúng
- ☐ Fallback `text_mode` hoạt động khi Gemini unavailable
- ☐ Graceful disconnect khi session kết thúc hoặc abandon
- ☐ Error handling cho mất kết nối giữa chừng
- ☐ Không bị memory leak (sessions map được dọn khi disconnect)

---END COPY---

SUB-ISSUE 4: [BE] Hệ thống chấm điểm (Grading & Evaluation)

Title

[BE] Hệ thống chấm điểm Speaking (Grading & Evaluation)

Labels

speaking, backend, priority:high

Body

---BEGIN COPY---

Mô tả

Xây dựng hệ thống chấm điểm Speaking bất đồng bộ sử dụng OpenRouter + Gemini multimodal model.

Parent issue: #__ Depends on: [BE] REST API (#__)

Flow chấm điểm

```
POST /sessions/{id}/complete
| -> SpeakingSessionService.complete()
```



```

|-> set is_finalized = true, status = "grading"
|-> trừ lúa
|-> evaluateSessionAsync(sessionId) <-- @Async
    |
    v
SpeakingEvaluationService (chạy background)
    |-> Load session + transcripts
    |-> Download audio từ Supabase Storage
    |-> Build grading prompt (audio + transcript + IELTS rubric)
    |-> Gọi OpenRouter API (Gemini multimodal model)
    |-> Parse JSON grading result
    |-> Lưu scores: overall, fluency, lexical, grammar, pronunciation
    |-> Lưu chi tiết vào grading_result JSONB
    |-> Set status = "graded", graded_at = now()
    |
    | (nếu lỗi)
    |-> Set status = "grading_failed"

```

Grading Result Schema (JSONB)

Lưu vào `speaking_sessions.grading_result`:

```

{
  "overall_band": 6.5,
  "criteria": {
    "fluency_coherence": {
      "band": 7.0,
      "feedback": "Good flow with occasional hesitations...",
      "highlights": ["natural transitions", "topic development"]
    },
    "lexical_resource": {
      "band": 6.5,
      "feedback": "Adequate vocabulary range...",
      "highlights": ["topic-specific vocabulary", "some repetition"]
    },
    "grammatical_range": {
      "band": 6.0,
      "feedback": "Mix of simple and complex structures...",
      "highlights": ["attempted complex sentences", "some errors"]
    },
    "pronunciation": {
      "band": 6.5,
      "feedback": "Generally clear pronunciation...",
      "highlights": ["clear stress patterns", "some L1 influence"]
    }
  },
  "per_part_feedback": {
    "part1": "Answered confidently with natural pace...",
    "part2": "Good development of topic, clear structure...",
    "part3": "Engaged with abstract ideas..."
  }
},

```

```
"improvement_tips": [  
  "Practice speaking for 2+ minutes on each topic",  
  "Use more complex sentence structures",  
  "Work on pronunciation of specific sounds"  
]  
}
```

Component: **SpeakingEvaluationService**

Method	Mô tả
<code>evaluateSessionAsync(Long sessionId)</code>	Entry point, <code>@Async</code> . Chạy <code>gradeSession</code> trong thread riêng
<code>gradeSession(Long sessionId)</code>	Logic chấm điểm chính
<code>getResults(Long sessionId, UUID userId)</code>	Trả về <code>SpeakingResultDTO</code>
<code>getGradingStatus(Long sessionId, UUID userId)</code>	Trả về <code>SpeakingGradingStatusDTO</code>
<code>buildGradingPrompt(...)</code>	Construct AI prompt với IELTS Speaking rubric
<code>parseGradingResult(String json)</code>	Parse OpenRouter response thành <code>SpeakingGradingResultDTO</code>

Prompt template (sketch)

```
You are an IELTS Speaking examiner. Evaluate the following speaking test.  
  
Session mode: {mode}  
Part(s) included: {parts}  
  
For each part, I will provide:  
- The examiner's question (text)  
- The candidate's audio response  
- The candidate's transcript (ASR)  
  
Evaluate using the official IELTS Speaking band descriptors:  
1. Fluency and Coherence (FC)  
2. Lexical Resource (LR)  
3. Grammatical Range and Accuracy (GRA)  
4. Pronunciation (P)  
  
Return a JSON object with the structure: { overall_band, criteria,  
per_part_feedback, improvement_tips }
```

Component: **SupabaseStorageService** (existing)

Sử dụng service có sẵn để download audio từ bucket `speaking-audio`.

Component: `OpenRouterClient` (existing)

Sử dụng client có sẵn. Cần hỗ trợ multimodal input (audio + text).

Configuration

```
speaking.evaluation.model=${SPEAKING_EVALUATION_MODEL:google/gemini-2.5-flash}
openrouter.api.key=${OPENROUTER_API_KEY:}
```

Acceptance Criteria

- ☐ Grading chạy async, không block response `/complete`
- ☐ `GET /grading-status` phản ánh đúng: `grading` -> `graded` hoặc `grading_failed`
- ☐ Kết quả có đủ 4 tiêu chí IELTS + overall band
- ☐ Kết quả lưu đúng: denormalized bands (columns) + JSONB detail
- ☐ Grading failure được xử lý gracefully
- ☐ Thời gian chấm < 60 giây cho session thông thường

---END COPY---

SUB-ISSUE 5: [FE] Core Flow - Pages, Routing, State Management

Title

```
[FE] Core flow - Pages, routing, state management
```

Labels

```
speaking, frontend, priority:high
```

Body

---BEGIN COPY---

Mô tả

Xây dựng frontend core cho Speaking: routing (PHẢI ACTIVE), Zustand store, pages, API client.

Parent issue: #__ **Depends on:** [BE] REST API (#__)

Routes

QUAN TRỌNG: Routes phải **ACTIVE**, KHÔNG redirect sang `/courses`. Code cũ có dòng `<Route path="/speaking/*" element={<Navigate to="/courses" replace />} />` -- phải XÓA.

Route	Component	Mô tả
-- (modal)	SpeakingPartModal	Modal chọn Part trên CourseDetailPage
/speaking/session/:mode	SpeakingSessionPage	Trang thi chính
/speaking/results/:sessionId	SpeakingResultsPage	Trang kết quả

Route registration trong App.jsx

```
{/* Speaking routes - ACTIVE */}
<Route path="/speaking/session/:mode" element={
  <ProtectedRoute><SpeakingSessionPage /></ProtectedRoute>
} />
<Route path="/speaking/results/:sessionId" element={
  <ProtectedRoute><SpeakingResultsPage /></ProtectedRoute>
} />
```

Pages

1. SpeakingPartModal (entry point)

- Hiển thị trên CourseDetailPage khi user chọn "Test Speaking"
- Cho phép chọn Part 1, 2, 3 riêng lẻ hoặc Full Test
- Navigate tới `/speaking/session/{mode}?topicId=...`
- File: `src/components/SpeakingPartModal.jsx`

2. SpeakingSessionPage (state machine)

Trang thi chính. Chuyển qua 5 state tuần tự:

```
INIT -> PRE_BRIEF -> LIVE_SESSION -> PROCESSING -> RESULTS
```

State	Render component	Mô tả
INIT	Loading spinner	Gọi API tạo session
PRE_BRIEF	PreBriefScreen	Cấp quyền mic, chọn accent/speed
LIVE_SESSION	GeminiLiveSessionLayout	Phần thi chính
PROCESSING	ProcessingScreen	Chờ AI chấm điểm
RESULTS	Navigate	Redirect tới <code>/speaking/results/{id}</code>

- File: `src/pages/speaking/SpeakingSessionPage.jsx`

3. SpeakingResultsPage

- Hiển thị overall band + 4 criteria bands (gauge/progress bars)
- Feedback chi tiết per criterion
- Per-part feedback
- Improvement tips
- Nếu chưa có kết quả: polling `GET /grading-status` mỗi 3-5 giây
- Nếu grading failed: hiển thị error + nút retry/back
- File: `src/pages/speaking/SpeakingResultsPage.jsx`

Zustand Store: `useSpeakingStore`

File: `src/stores/useSpeakingStore.js`

State

```
{
  // Session
  sessionId: null,
  sessionMode: null,      // 'full', 'part1', 'part2', 'part3'
  topicId: null,
  questions: [],
  currentQuestionIndex: 0,
  phase: 'idle',          // idle | init | pre_brief | live | processing |
  completed

  // Config
  accent: 'neutral',      // british | american | australian | neutral
  speed: 'normal',        // slow | normal | fast

  // Transcript data
  transcripts: {},         // { [questionId]: { text, audioUrl, duration } }
  currentTranscript: '',  // Live transcript đang gõ

  // Results
  results: null,           // SpeakingResultDTO
  gradingStatus: null,     // 'grading' | 'graded' | 'grading_failed'

  // UI state
  loading: false,
  error: null,
}
```

Actions

```
// Session lifecycle
createSession(mode, topicId, accent, speed) // POST /sessions
startQuestion()                             // Advance currentQuestionIndex
saveTranscript(questionId, audioUrl, text)  // POST
```

```

/sessions/{id}/transcripts
completeSession() // POST /sessions/{id}/complete
abandonSession() // POST /sessions/{id}/abandon

// Results
pollGradingStatus() // GET /sessions/{id}/grading-
status
getResults() // GET /sessions/{id}/results

// Utility
reset() // Clear all state
setAccent(accent)
setSpeed(speed)

```

API Client: `speakingApi.js`

File: `src/api/speakingApi.js`

```

import { api } from './backendApi';

export const speakingApi = {
  // Content
  getTopics: () => api.get('/speaking/topics'),
  getQuestions: (topicId, mode) =>
    api.get('/speaking/questions', { params: { topicId, mode } }),

  // Sessions
  createSession: (data) => api.post('/speaking/sessions', data),
  getSession: (id) => api.get(`/speaking/sessions/${id}`),
  saveTranscript: (id, data) =>
    api.post(`/speaking/sessions/${id}/transcripts`, data),
  completeSession: (id) => api.post(`/speaking/sessions/${id}/complete`),
  abandonSession: (id) => api.post(`/speaking/sessions/${id}/abandon`),

  // Grading
  getGradingStatus: (id) => api.get(`/speaking/sessions/${id}/grading-status`),
  getResults: (id) => api.get(`/speaking/sessions/${id}/results`),

  // History
  getHistory: (params) => api.get('/speaking/history', { params }),
};

export function setupSpeakingApiClient(apiInstance) {
  // Share the configured axios instance from backendApi
}

```

CSS files

File

Mục đích

File	Mục đích
src/css/speaking/speaking-session.css	Session page layout
src/css/speaking/speaking-results.css	Results page styles
src/css/speaking/speaking-components.css	Shared component styles

Acceptance Criteria

- ☐ Routes hoạt động (KHÔNG redirect sang /courses)
- ☐ SpeakingPartModal mở đúng, navigate tới session page
- ☐ State machine chuyển đúng qua 5 phases
- ☐ Results page hiển thị đầy đủ khi có data
- ☐ Polling grading status hoạt động (3-5s interval)
- ☐ Error states hiển thị đúng (grading_failed, network error)
- ☐ Store reset đúng khi quay lại CourseDetailPage
- ☐ Responsive trên mobile (mic button phải dễ bấm)

---END COPY---

SUB-ISSUE 6: [FE] Real-time Session - Audio & WebSocket

Title

[FE] Real-time session - Audio recording & WebSocket

Labels

speaking, frontend, priority:high

Body

---BEGIN COPY---

Mô tả

Xây dựng UI real-time speaking session: kết nối WebSocket, thu âm, phát audio examiner, ASR, các cơ chế phát hiện tự động.

Parent issue: #__ Depends on: [BE] WebSocket (#), [FE] Core flow (#)

Components

1. GeminiLiveSessionLayout -- Layout chính phần thi

File: `src/components/speaking/GeminiLiveSessionLayout.jsx`

Hiển thị:

- Câu hỏi hiện tại (text, luôn hiện)
- Examiner audio player (auto-play khi nhận audio từ WS)
- `ExaminerWaveform` -- visualization khi AI đang nói
- Transcript real-time (cập nhật liên tục từ WS)
- Nút microphone (sáng = đang thu, tắt = chưa thu)
- `TimeWarningBadge` -- cảnh báo gần hết giờ
- Nút "Tiếp theo" / "Submit"

2. `PreBriefScreen` -- Màn hình chuẩn bị

File: `src/components/speaking/PreBriefScreen.jsx`

- Hiển thị topic, test info
- Request microphone permission (`navigator.mediaDevices.getUserMedia({ audio: true })`)
- Dropdown chọn accent & speed
- Nút "Bắt đầu buổi thi"
- Hiện warning nếu user từ chối mic permission

3. `Part2PrepLayout` -- Chuẩn bị Part 2

File: `src/components/speaking/Part2PrepLayout.jsx`

- Hiển thị cue card + bullet points
- Timer đếm ngược 1 phút chuẩn bị
- Nút "Bắt đầu nói" (hoặc tự chuyển khi hết 1 phút)

4. `ProcessingScreen` -- Chờ chấm điểm

File: `src/components/speaking/ProcessingScreen.jsx`

- Loading animation
- Progress steps: "Đang upload audio..." -> "Đang phân tích..." -> "Đang chấm điểm..."
- Auto-redirect khi `gradingStatus === 'graded'`

5. `ExaminerWaveform` -- Visualization audio

File: `src/components/speaking/ExaminerWaveform.jsx`

- Animated waveform khi examiner đang phát audio
- Ẩn/dừng khi `examiner_speaking = false`

6. `TimeWarningBadge`

File: `src/components/speaking/TimeWarningBadge.jsx`

- Hiện badge cảnh báo khi còn < 30 giây

Hooks

useGeminiLive(sessionId, options) -- WebSocket manager

File: `src/hooks/useGeminiLive.js`

```

const {
  isConnected,          // WebSocket connected
  isReady,              // Gemini setup complete
  isExaminerSpeaking,   // AI đang đọc câu hỏi
  examinerTranscript,   // Text AI đang nói
  userTranscript,        // Text user đang nói (ASR từ Gemini)
  error,

  connect,              // Mở WS connection
  disconnect,           // Đóng WS connection
  sendStartQuestion,    // Gửi {type: "start_question"}
  sendEndTurn,          // Gửi {type: "end_turn"}
  sendAudioChunk,       // Gửi binary audio frame
} = useGeminiLive(sessionId, {
  onExaminerAudio: (audioData) => { /* play audio */ },
  onExaminerSpeaking: (isSpeaking) => { /* toggle mic */ },
  onTranscript: (text, source) => { /* update UI */ },
  onTurnComplete: () => { /* advance question */ },
  onError: (error) => { /* handle */ },
  onFallback: () => { /* switch to text-only mode */ },
});

```

Internals:

- Connect tới `ws[s]://host/ws/speaking/{sessionId}`
- Handle message types: `status`, `examiner_audio` (binary), `transcript`, `examiner_speaking`, `turn_complete`, `error`
- Forward binary audio chunks từ mic
- Auto-reconnect on unexpected disconnect (max 3 retries)

useAudioRecorder() -- Thu âm microphone

File: `src/hooks/useAudioRecorder.js`

```

const {
  isRecording,
  audioBlob,            // Final blob sau khi stop
  startRecording,
  stopRecording,
  onDataAvailable,      // Callback cho mỗi audio chunk (streaming)
} = useAudioRecorder();

```

Internals:

- Sử dụng `MediaRecorder` API

- Format: `audio/webm;codecs=opus` (hoặc fallback)
- `timeslice` param để nhận chunks mỗi 250ms cho streaming
- Khi stop: tạo final `Blob` để upload lên Supabase Storage

`useSilenceDetection(options)` -- Phát hiện im lặng

File: `src/hooks/useSilenceDetection.js`

```
const { isSilent, silenceDuration } = useSilenceDetection({
  stream: mediaStream,      // Audio stream từ getUserMedia
  threshold: -50,           // dB threshold
  silenceTimeout: 3000,     // ms trước khi trigger
  onSilenceDetected: () => { /* auto-advance question */ },
});
```

Internals:

- Tạo `AudioContext` + `AnalyserNode`
- Monitor audio input level via `getByteFrequencyData`
- Khi âm lượng < threshold liên tục > `silenceTimeout` ms -> trigger callback
- Dùng để tự chuyển câu khi user ngừng nói

`useBargeInDetection(options)` -- Phát hiện nói xen

File: `src/hooks/useBargeInDetection.js`

```
const { isBargeIn } = useBargeInDetection({
  stream: mediaStream,
  isExaminerSpeaking: true, // Chỉ monitor khi AI đang nói
  threshold: -40,
  onBargeIn: () => { /* stop examiner audio, open mic */ },
});
```

Internals:

- Monitor audio input level khi examiner đang speaking
- Nếu phát hiện user nói (vượt threshold) -> interrupt examiner
- Tắt khi `isExaminerSpeaking = false`

`useRealtimeASR()` -- Fallback ASR (optional)

File: `src/hooks/useRealtimeASR.js`

```
const {
  transcript, isProcessing,
  transcribeAudio, // (audioBlob) => transcript text
}
```

```
    checkStatus,          // Check ASR service availability
  } = useRealtimeASR();
```

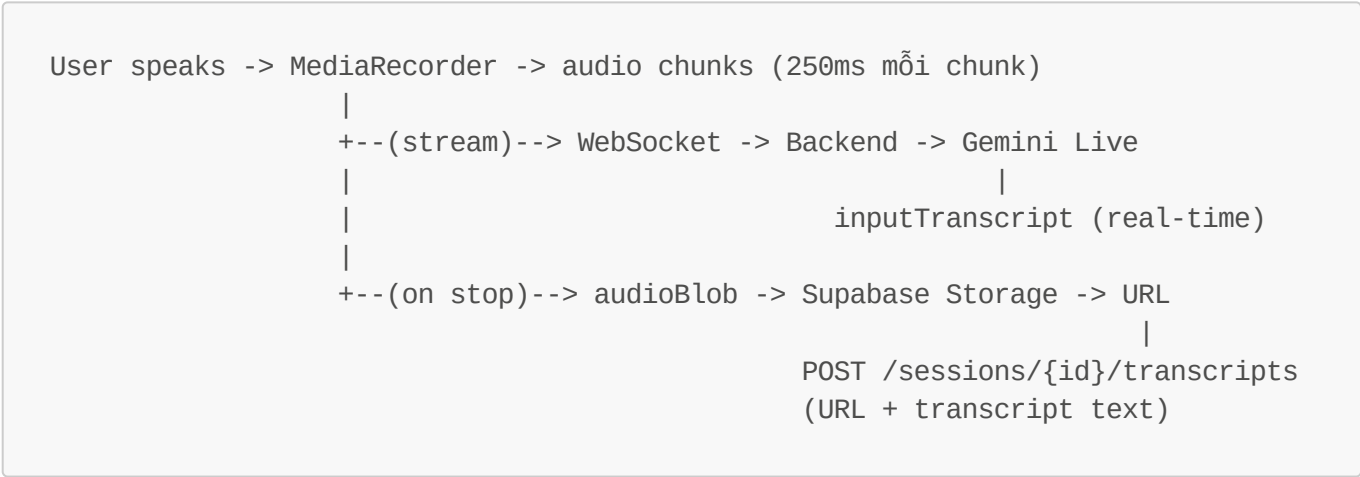
Sử dụng khi Gemini Live transcript không đủ tốt. Gọi `POST /api/speaking/transcribe`.

`useTimer(durationSeconds)` -- Đếm ngược

File: `src/hooks/useTimer.js`

```
const {
  timeLeft,      // Seconds remaining
  isRunning,
  isExpired,     // timeLeft === 0
  start, pause, reset,
} = useTimer(120); // 2 phút cho Part 2
```

Audio Flow tổng thể



Fallback behavior

Tình huống	Hành vi
Gemini trả <code>status=fallback_text_mode</code>	Ẩn audio controls. Chỉ hiện text câu hỏi. User vẫn nói, audio vẫn record & upload.
Examiner audio chậm > 3 giây	Tự động mở mic, user trả lời dựa trên text
Upload audio lỗi	Try-catch, log warning, session tiếp tục
ASR lỗi liên tục	Chuyển sang <code>useRealtimeASR</code> fallback (nếu available)

CSS files

File	Mục đích
<code>src/css/speaking/speaking-live.css</code>	Live session layout

File	Mục đích
src/css/speaking/speaking-components.css	Shared components
src/css/speaking/examiner-waveform.css	Waveform animation
src/css/speaking/conversation-player.css	Conversation replay

Acceptance Criteria

- ☐ Microphone permission request hoạt động trên Chrome, Firefox, Safari
- ☐ Audio recording và streaming qua WebSocket (chunks mỗi 250ms)
- ☐ Examiner audio phát tự động, waveform hiển thị đúng
- ☐ Transcript real-time cập nhật liên tục trên UI
- ☐ Silence detection tự chuyển câu sau 3s im lặng
- ☐ Barge-in detection interrupt examiner khi user nói xen
- ☐ Part 2 preparation timer 1 phút hoạt động
- ☐ Fallback text-only mode hoạt động khi Gemini unavailable
- ☐ Upload audio lên Supabase Storage thành công
- ☐ Không bị audio echo/feedback loop

---END COPY---

SUB-ISSUE 7: [Infra] Cleanup Job, Fallback & Kiểm thử

Title

[Infra] Cleanup job, fallback mechanisms & kiểm thử

Labels

speaking, infrastructure, priority:medium

Body

---BEGIN COPY---

Mô tả

Implement các thành phần infra: cleanup job dọn audio rác (MỚI, chưa từng tồn tại), fallback mechanisms bổ sung, và integration testing.

Parent issue: #__ **Depends on:** Tất cả sub-issues khác

1. SpeakingAudioCleanupJob (MỚI)

LƯU Ý: Job này được mô tả trong design doc nhưng **chưa bao giờ được implement**. Đây là code hoàn toàn mới.

Spec

- Class: `com.cramer.job.SpeakingAudioCleanupJob`
- Chạy mỗi **6 giờ**: `@Scheduled(fixedRate = 6 * 60 * 60 * 1000)`
- Chỉ kích hoạt khi `speaking.cleanup.enabled = true`
- Sử dụng `@ConditionalOnProperty` hoặc check trong method

Logic

1. Query `speaking_sessions` WHERE:
 - `is_finalized = false`
 - `status = 'in_progress'`
 - `created_at < now() - {expire-hours}h`
2. Với mỗi session tìm được:
 - a. Load transcripts có `audio_url` NOT NULL
 - b. Xóa audio files trên Supabase Storage (bucket: `speaking-audio`)
 - c. Set `session.status = 'expired'`
 - d. Log: `"Expired session {id}, cleaned {n} audio files"`
3. Log tổng kết: `"Cleanup complete. Expired {n} sessions, deleted {m} audio files."`

Safety rules

- **KHÔNG** xóa sessions đã `is_finalized = true`
- **KHÔNG** xóa sessions có `status = 'completed'` hoặc `'graded'`
- **KHÔNG** xóa sessions tạo < `expire-hours` trước (user có thể quay lại)
- Có thể **tắt** trong dev/test bằng config
- **Log mỗi lần chạy** dù có xóa hay không

Configuration

```
speaking.cleanup.enabled=${SPEAKING_CLEANUP_ENABLED:true}  
speaking.cleanup.expire-hours=${SPEAKING_CLEANUP_EXPIRE_HOURS:24}
```

Dependencies

- `SpeakingSessionRepository` -- query sessions hết hạn
- `SpeakingTranscriptRepository` -- lấy audio URLs
- `SupabaseStorageService` -- xóa files

2. Fallback mechanisms bổ sung

2a. ASR Fallback

Khi Gemini Live không trả transcript (hoặc trả transcript rỗng liên tục > 3 câu):

- Frontend phát hiện: `userTranscript` rỗng sau `turnComplete`
- Tự chuyển sang dùng `useRealtimeASR` hook
- Hook gọi `POST /api/speaking/transcribe` với audio blob
- Backend dùng `RealtimeASRService` (OpenRouter) để transcribe

2b. Grading retry

- Nếu grading thất bại (`status = "grading_failed"`):
 - Frontend hiện nút "Thử lại"
 - Nút gọi lại `POST /sessions/{id}/complete` (hoặc endpoint retry riêng)
 - Backend re-trigger `evaluateSessionAsync`
 - KHÔNG trừ thêm lúa (đã trừ rồi)

2c. Session timeout

- Backend: `@Scheduled` job (hoặc tích hợp vào cleanup job)
- Sessions có `status = 'in_progress'` và không có activity > 30 phút
- Tự set `status = 'abandoned'`
- KHÔNG trừ lúa

3. Kiểm thử

Backend unit tests

- ☐ `SpeakingSessionServiceTest`
 - Tạo session thành công khi đủ lúa
 - Tạo session thất bại khi hết lúa
 - Complete session: `is_finalized = true`, `status = grading`, lúa bị trừ
 - Abandon session: `status = abandoned`, lúa KHÔNG bị trừ
 - Complete session không trừ lúa 2 lần (lúa_deducted guard)
- ☐ `SpeakingEvaluationServiceTest`
 - Grading thành công: scores lưu đúng
 - Grading thất bại: `status = grading_failed`
 - Mock OpenRouter response
- ☐ `SpeakingAudioCleanupJobTest`
 - Chỉ expire sessions hết hạn (> 24h, `is_finalized = false`)
 - Không expire sessions mới tạo
 - Không expire sessions đã finalized
 - Disabled khi `config = false`

Backend integration tests

- ☐ Full flow qua REST endpoints: tạo session -> lưu transcripts -> complete -> get results
- ☐ WebSocket connection test: connect, send/receive messages, disconnect
- ☐ Credit flow: kiểm tra balance trước/sau session

Frontend tests (nếu có test setup)

- ☐ **SpeakingPartModal**: render, chọn part, navigate đúng URL
- ☐ **useSpeakingStore**: state transitions (idle -> init -> live -> processing -> completed)
- ☐ **useTimer**: countdown, pause, reset, expired state

Manual testing checklist

- ☐ **Happy path**: Chọn topic -> chọn part -> cấp mic -> thi -> submit -> xem kết quả
- ☐ **Full test**: Thi cả 3 part liên tiếp
- ☐ **Abandon**: Thi dở -> thoát -> lúa KHÔNG bị trừ
- ☐ **Fallback**: Tắt Gemini Live config -> text-only mode hoạt động
- ☐ **Credit gate**: Hết lúa -> không tạo được session (báo lỗi rõ ràng)
- ☐ **Results persistence**: Reload trang results -> data vẫn hiển thị
- ☐ **Grading wait**: Submit -> loading screen -> tự chuyển khi có kết quả
- ☐ **Mobile**: Test trên mobile browser (mic button dễ bấm, layout không vỡ)
- ☐ **Network error**: Mất mạng giữa chừng -> error message hiển thị đúng

4. Vertex AI Service Account (Production)

Cho production deployment cần:

- ☐ Tạo Google Cloud project (hoặc dùng existing)
- ☐ Tạo Service Account với Vertex AI permissions
- ☐ Cấu hình authentication (service account key hoặc workload identity)
- ☐ Chuyển từ API key trực tiếp sang service account
- ☐ Test connection qua Vertex AI endpoint thay vì Google AI endpoint
- Tham khảo: [Vertex AI Live API](#)

Acceptance Criteria

- ☐ Cleanup job chạy đúng schedule, dọn đúng files, không xóa nhầm
- ☐ ASR fallback hoạt động khi Gemini transcript lỗi
- ☐ Grading retry không trừ thêm lúa
- ☐ Session timeout auto-abandon hoạt động
- ☐ Tất cả unit tests pass
- ☐ Manual testing checklist hoàn thành 100%

---END COPY---

HƯỚNG DẪN SỬ DỤNG GITHUB ISSUES (cho người mới)

Bước 1: Tạo Labels

Vào repo trên GitHub -> tab **Issues** -> **Labels** -> **New label**

Tạo các labels sau:

Label	Color (gợi ý)	Mô tả
/		

Label	Color (gợi ý)	Mô tả
speaking	#a855f7 (tím)	Speaking feature
feature	#0e8a16 (xanh lá)	New feature
backend	#1d76db (xanh dương)	Backend work
frontend	#fbca04 (vàng)	Frontend work
database	#e4e669 (vàng nhạt)	Database changes
infrastructure	#d4c5f9 (tím nhạt)	Infra/DevOps
priority:high	#b60205 (đỏ)	High priority
priority:medium	#ff9f1c (cam)	Medium priority

Bước 2: Tạo Milestone (tùy chọn)

Tab **Issues** -> **Milestones** -> **New milestone**

- Title: **Speaking MVP**
- Due date: (chọn deadline nếu có)
- Description: "Ship tính năng Speaking practice với AI examiner"

Bước 3: Tạo Parent Issue

1. Tab **Issues** -> **New issue**
2. **Title**: copy dòng trong phần **### Title** của PARENT ISSUE ở trên
3. **Body**: copy nội dung giữa **---BEGIN COPY---** và **---END COPY---**
4. Thêm labels: **speaking, feature, priority:high**
5. Thêm milestone: **Speaking MVP** (nếu đã tạo)
6. **Submit new issue** -> ghi nhớ số issue (vd: **#10**)

Bước 4: Tạo các Sub-issues

Lặp lại cho mỗi sub-issue (7 cái):

1. **New issue**
2. Copy Title và Body từ section tương ứng
3. Thêm labels phù hợp
4. Thêm milestone
5. **Submit** -> ghi nhớ số issue

Bước 5: Liên kết Sub-issues vào Parent

1. Quay lại parent issue (vd: **#10**)
2. Nhấn **Edit** trên body
3. Thay **#__** bằng số issue thật:

- [] #11 -- [DB] Thiết kế schema & migration cho Speaking
- [] #12 -- [BE] REST API - Quản lý session Speaking
- [] #13 -- [BE] WebSocket + Gemini Live API integration
- [] #14 -- [BE] Hệ thống chấm điểm (Grading & Evaluation)
- [] #15 -- [FE] Core flow - Pages, routing, state management
- [] #16 -- [FE] Real-time session - Audio & WebSocket
- [] #17 -- [Infra] Cleanup job, fallback & kiểm thử

4. **Update** -> GitHub tự render progress bar

Bước 6: Liên kết PR -> Issue (khi code xong)

Trong PR description, viết:

Closes #12

Khi merge PR, issue #12 sẽ **tự động đóng** và checkbox trong parent issue tự tick.

Tips hữu ích

- **Filter issues:** Dùng `is:open label:speaking label:backend` để lọc
- **Pin issue:** Click "..." trên parent issue -> "Pin issue" để ghim lên đầu
- **Assign:** Click "Assignees" để gán người phụ trách
- **Comment:** Viết comment trên issue khi có update quan trọng
- **Projects:** Dùng tab "Projects" nếu muốn kanban board (tùy chọn, không bắt buộc)
- **Close vs Complete:** Khi merge PR với `Closes #N`, issue tự đóng. Hoặc đóng thủ công bằng nút "Close issue"

Thứ tự tạo issues khuyến nghị

1. Tạo parent issue trước
2. Tạo sub-issues theo thứ tự dependency:
 - #1 [DB] -> #2 [BE REST] -> #3 [BE WS] -> #4 [BE Grading]
 - > #5 [FE Core] -> #6 [FE Realtime]
 - > #7 [Infra]
3. Quay lại parent, link tất cả sub-issues
4. Assign people (nếu biết ai làm gì)